Build Task — Retriever Module — Hints & Solution

> Back to the Build Task · Hint 1 · Hint 2 · Solution Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried.

- Hint 1 — What you're building, and the exact pieces
- Hint 2 — The plan, and almost the whole thing
- Solution

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 1 — What you're building, and the exact pieces

The same files, written the way the exercises wrote them:

- **`Path(folder).glob("*")` and `path.suffix`** — from the Real-world exercise, to keep only `.txt` / `.md` files.
- **`chunk_fn: Callable[[str], list[str]]`** — from the Intermediate exercise: the chunker is a parameter, so `build_vector_store(folder, chunk_by_chars)` swaps the method without editing `ingest.py`.
- **One id per chunk, built from the file and position:** `f"{path.name}-{chunk_index}"`. The same file always gives the same ids, never random ones.
- **Embeddings in batches** — from the Failure exercise: one call per 1,000 texts, never one call per chunk.
- **A clean rebuild** — from the Real-world exercise: `delete_collection()`, catching `chromadb.errors.NotFoundError` on the first run, then `create_collection()`.
- **LangChain's `Chroma`** — from the Real-world exercise: `OpenAIEmbeddings(model=...)`, `Chroma(collection_name=..., persist_directory=...)`, `delete_collection()`, `Chroma.from_texts(...)` and `similarity_search_with_score()` (each result has `.page_content` and `.metadata`).
- **`test_retriever.py` with PASS / FAIL** — each case checks its own result, instead of you reading every line.

Think past "ingest once and it works," too. Ask: **what happens if `build_vector_store()` runs a second time on the same folder?** With a plain `create_collection()`, it crashes — the collection already exists. And `collection.add()` with an id that's already stored is silently ignored, so an

edited file would keep its old text. The answer is the clean rebuild: delete the collection and build it again from the files as they are now.

A second question: what happens when `doc_folder` has thousands of files? One embedding call per chunk is slow. Collect the chunks from *all* files first, then embed them in a few large batches.

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

`chunking.py` — the same two functions, with type hints.

`ingest.py`:

```
function load_chunks(doc_folder, chunk_fn) -> (texts, metadatas, ids):
    for each path in Path(doc_folder).glob("*"):
        skip it unless path.suffix is ".txt" or ".md"
        chunks = chunk_fn(path.read_text())
        for chunk_index, chunk_text in enumerate(chunks):
            add chunk_text, {source_file, chunk_index},
                and the id f"{path.name}-{chunk_index}"
    return the three lists

function build_vector_store(doc_folder, chunk_fn=chunk_by_paragraph,
                            collection_name="docs"):
    delete the old collection (NotFoundError on the first run is fine)
    create it again
    texts, metadatas, ids = load_chunks(doc_folder, chunk_fn)
    if no texts: return the empty collection
    embed the texts in batches of 1,000
    add everything in one collection.add(...) call
    return the collection
```

`retriever.py` — the Basic plan, with type hints and `zip()` over the three result lists.

One piece to get you unstuck — collecting the chunks, with their metadata and stable ids:

```
def load_chunks(
    doc_folder: str, chunk_fn: Callable[[str], list[str]],
) -> tuple[list[str], list[dict], list[str]]:
    all_texts: list[str] = []
    all_metadatas: list[dict] = []
    all_ids: list[str] = []
```

```
    for path in Path(doc_folder).glob("*"):
        if path.suffix not in DOC_SUFFIXES:
            continue
        # your turn: chunk the file's text with chunk_fn, then append
        # each chunk's text, its metadata dict and its id
        ...
    return all_texts, all_metadatas, all_ids
```

The empty-store check has to happen *before* you search — searching an empty store is exactly the case the requirements call out as "must not crash."

Then sketch the clean rebuild and the batched embedding call yourself — both are in the exercises (the Real-world and Failure sections) — before checking the Solution.

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Solution

Story — `ingest.py` (Intermediate): `load_chunks()` reads and chunks the files (from the Real-world exercise), and `chunk_fn` is typed as `Callable[[str], list[str]]` (from the Intermediate exercise). Every id is built from the file and position. **If not:** reading files and talking to Chroma would be tangled in one long function, and nothing would say what kind of function `chunk_fn` must be.

`chunking.py` gets type hints (`text: str` , `-> list[str]`); nothing else changes.

```
# ingest.py
from pathlib import Path
from typing import Callable

import chromadb
from openai import OpenAI

from chunking import chunk_by_paragraph
from config import load_config
from logging_setup import get_logger

DOC_SUFFIXES = (".txt", ".md")

chroma_client = chromadb.Client()

def load_chunks(
    doc_folder: str, chunk_fn: Callable[[str], list[str]],
) -> tuple[list[str], list[dict], list[str]]:
```

```
    all_texts: list[str] = []
    all_metadatas: list[dict] = []
    all_ids: list[str] = []
    for path in Path(doc_folder).glob("*"):
        if path.suffix not in DOC_SUFFIXES:
            continue
        chunks = chunk_fn(path.read_text())
        for chunk_index, chunk_text in enumerate(chunks):
            all_texts.append(chunk_text)
            all_metadatas.append({
                "source_file": path.name, "chunk_index": chunk_index,
            })
            # why: the same file and position always give the same id
            all_ids.append(f"{path.name}-{chunk_index}")
    return all_texts, all_metadatas, all_ids

def build_vector_store(
    doc_folder: str,
    chunk_fn: Callable[[str], list[str]] = chunk_by_paragraph,
    collection_name: str = "docs",
):
    config = load_config()
    logger = get_logger(__name__)
    collection = chroma_client.create_collection(name=collection_name)

    all_texts, all_metadatas, all_ids = load_chunks(doc_folder, chunk_fn)
    if not all_texts:
        logger.warning(f"no documents found in {doc_folder}")
        return collection

    client = OpenAI()
    response = client.embeddings.create(
        model=config.embedding_model, input=all_texts,
    )
    embeddings: list[list[float]] = []
    for item in response.data:
        embeddings.append(item.embedding)

    collection.add(
        ids=all_ids, embeddings=embeddings,
        documents=all_texts, metadatas=all_metadatas,
    )
    logger.info(f"ingested {len(all_texts)} chunks from {doc_folder}")
    return collection
```

```
# retriever.py
from openai import OpenAI

from config import load_config
from logging_setup import get_logger

def retrieve(collection, query: str, k: int = 3) -> list[dict]:
    logger = get_logger(__name__)
    if collection.count() == 0:
        # why: an empty store must return [], not crash (Requirements)
        logger.info(f"empty store, no results for: {query}")
        return []

    config = load_config()
    client = OpenAI()
    response = client.embeddings.create(
        model=config.embedding_model, input=[query],
    )
    query_embedding = response.data[0].embedding

    results = collection.query(query_embeddings=[query_embedding], n_results=k)
    texts = results["documents"][0]
    metadatas = results["metadatas"][0]
    distances = results["distances"][0]

    documents: list[dict] = []
    for text, metadata, distance in zip(texts, metadatas, distances):
        documents.append({
            "text": text,
            "source_file": metadata["source_file"],
            "chunk_index": metadata["chunk_index"],
            "score": distance,
        })

    logger.info(f"query {query!r}: {len(documents)} results")
    return documents
```

Story — `test_retriever.py` (Intermediate): each case now checks its own result and prints PASS or FAIL. **If not:** you'd have to read every printed line and decide yourself whether it's right.

```
# test_retriever.py
# Runs the 4 Test Cases from the README and prints PASS or FAIL for each.
# Run it from inside this folder: python test_retriever.py
# Makes real API calls — needs a valid OPENAI_API_KEY in .env.
from pathlib import Path
```

```
from ingest import build_vector_store
from retriever import retrieve

def report(name: str, passed: bool, detail: str) -> None:
    if passed:
        print(f"PASS  {name} -> {detail}")
    else:
        print(f"FAIL  {name} -> {detail}")

def test_word_for_word(collection) -> None:
    query = "Store coffee beans in an airtight container, away from light."
    top = retrieve(collection, query)[0]["source_file"]
    report("case 1 (word-for-word)", top == "Storing coffee beans.txt", top)

def test_reworded(collection) -> None:
    results = retrieve(collection, "How long do ground beans stay good?")
    sources: list[str] = []
    for result in results:
        sources.append(result["source_file"])
    passed = "Storing coffee beans.txt" in sources
    report("case 2 (reworded)", passed, f"top 3: {sources}")

def test_off_topic(collection) -> None:
    on_topic = retrieve(collection, "Which grind fits a French press?", k=1)
    off_topic = retrieve(collection, "How do I train for a marathon?", k=1)
    on_score = on_topic[0]["score"]
    off_score = off_topic[0]["score"]
    # how: a distance, so the off-topic best match must be FARTHER away
    passed = off_score > on_score
    detail = f"distance {off_score:.2f} vs on-topic {on_score:.2f}"
    report("case 3 (off-topic)", passed, detail)

def test_empty_store() -> None:
    Path("empty_docs").mkdir(exist_ok=True)
    empty = build_vector_store("empty_docs", collection_name="empty")
    results = retrieve(empty, "anything")
    report("case 4 (empty store)", results == [], str(results))

if __name__ == "__main__":
    collection = build_vector_store("docs")
    test_word_for_word(collection)
    test_reworded(collection)
    test_off_topic(collection)
    test_empty_store()
```

Expected output (your distances will differ):

```
... ingest WARNING no documents found in empty_docs
PASS  case 1 (word-for-word) -> Storing coffee beans.txt
PASS  case 2 (reworded) -> top 3: ['Storing coffee beans.txt', ...]
PASS  case 3 (off-topic) -> distance 1.78 vs on-topic 1.14
PASS  case 4 (empty store) -> []
```